

Aristotle University of Thessaloniki
School of informatics

Distributed computation engine

Theodoros Kaimasidis

Supervisor: Petros Nicopolitidis

September 13, 2026

Abstract

This work is trying to solve a fundamental issue, that of combining many computing systems in order to create one of larger capacity, without however creating something that is embarrassingly-parallel. The exposure to the underlying mechanisms of the systems involved is also a welcome challenge, and with this I want to distill experience that would be hard to amass in a professional setting with the exciting field of research in the area of massive computation. Inspiration is drawn from the distributed systems applicable to commercial applications and software, as well as the more scientific endpoints of linux and the fundamental technologies used. The result resembles a RPC framework, with reusable and extensible components that for the most part bond well to the underlying structure of the orchestrator that manages the lifecycles of the userspace programs we are going to be using to have run the set of computations, of which results we are going to base ourselves on for the evaluation.

Contents

Abstract	i
1 Introduction	1
1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?	1
1.2 The hard part	1
1.3 Structure of this Thesis	2
2 Containers, Orchestration, and Scaling	3
2.1 Docker and the container model	3
2.2 Kubernetes: the orchestration model	6
2.2.1 kube-apiserver (The Central Hub)	6
2.2.2 etcd	7
2.2.3 kube-scheduler	7
2.2.4 Cluster Autoscaler / Karpenter	7
2.2.5 Worker Node	8
2.2.6 kubelet	8
2.2.7 Container Runtime	8
2.2.8 kube-proxy	9
2.2.9 How the Kinds Relate	9
2.2.10 Pod	9
2.2.11 ReplicaSet	10
2.2.12 Deployment	10
2.2.13 StatefulSet	11
2.2.14 DaemonSet	12
2.2.15 Job	12
2.2.16 CronJob	13
2.2.17 Service	13
2.2.18 ConfigMap & Secret	14
2.3 The object model	15
2.4 API gateways, ingress, and service mesh	15
3 Two Transports: XDP and gRPC	16
3.1 XDP: programmable packet processing at the driver	16
3.1.1 eBPF, maps, and the verifier	17
3.1.2 Attachment modes and the reality of zero copy	17
3.1.3 AF_XDP: the UMEM and its four rings	17
3.1.4 How this project uses XDP	17
3.1.5 Frame ownership across the pool, and in-kernel admission	18
3.1.6 Deployment: privileges and containers	18
3.2 gRPC: typed RPC on HTTP/2	18

3.3	Where each fits	19
4	Protocol Buffers	20
4.1	The schema and code generation	20
4.2	The wire format	20
4.3	Schema evolution	21
4.4	Implementing the wire format	21
4.4.1	Forward compatibility is one line	21
4.4.2	Default elision, and what it costs	22
4.4.3	The fiddly parts	22
4.4.4	Checking the claim	23
4.5	gRPC and Protocol Buffers together	23

Chapter 1

Introduction

This thesis is about computation, systems, distributed computation and their intertwining in an interesting project. The need is accelerated computation via the means of one or more computers/computation nodes, which may have lost some of its importance in our days, thanks to many available multi-core consumer and server computer processors, but it's one aspect of computing systems that's extremely rewarding and has too much to teach in the process of its implementation. The problem visited is a personal question of mine from when I was way younger, how can I combine two slow machines to run something faster.

1.1 How can we combine two or more computers to accelerate computation of one or multiple tasks?

This is something that many pc enthusiasts dream of doing, combining the power of multiple cpus or computers for more compute, and it's actually been a huge trend with the advent of data centers and distributed systems evolution and adoption on the business of software development and deployment. The case reviewed here is one about normal pcs with clear boundaries and store bought interconnects (Ethernet) facilitating their intercommunication. The first challenge met is the design of the system, unlike the 'system-design' paradigm of today's development circles however the architecture here can scale up and down, traverse and live in different layers. For example computation is discussed in the context of an os, also in the process context of the C runtime and also with the linux operating system being open source we have access to both with one tool: the C programming language. Further, concurrency is a "solved" issue, the problem is the communication and mutual exclusion of what constitutes a thread in our application, it's been a struggle designing something that is not a mere "embarrassing-parallel" machine, while it's been a similar note for the application of the distributed systems side of algorithms for consensus and communication between nodes.

1.2 The hard part

The hard part comes before the implementation, the decision of what to focus on, what to work on, what to reuse and what to write ourselves. The result of this work is software that can run on two or more machines and via a gateway obfuscate the raw computing power of both to provide higher compute than each of the constituents, hopefully in a scalable way (and in a scalable way that makes sense). For a start we need to define the "computation", what exactly is going to be worked up on: the choice has been made for this to be a set of input data and a function that's to run on them. This comes instead of a process, which in linux is completely open to a developer to see, even if there needs to be a kernel loadable module. Furthermore, this pair constitutes a "job", and the objective is to be able to distribute them in the environments that we control. A

plain worker written in C or Java for example would not suffice here, it's clear that there is a need of a head and a worker component on each installation. As a result after the choice of the execution mechanism, what is needed is a routing algorithm, or a choice of a routing strategy in order to be able to provide an actual higher capacity for computations across two systems, which in our finalized project is taken care of the head (written in C) that constitutes a distributed system. And finally, the landing ground for these has been set to be the kubernetes orchestrator engine, which means packaging in containers and managing of the compute nodes accordingly, this choice has been made because the philosophy of this technology perfectly contributes to the idea of the pooling together of the resources of multiple computers and also is in and of itself an interesting endeavor since much of the modern infrastructure operates in the form of a kubernetes cluster. What has been clear from the start of the project however is the same as what struck me most when I first studied concurrency, running threads in parallel is easy, initiating multiple processes is also easy, problems arise when we have exclusivity and in this way bottlenecks.

1.3 Structure of this Thesis

Chapter ?? covers ..., Chapter ?? presents ..., and Chapter ?? reports ...

This thesis puts into context the software project that is the showcase of the work. In total it explains the technologies used, explores the underlying concepts and the mechanisms that are exposed via apis or the ones that are applicable to the project in a meaningful way. A more general note of exploration and care has been given because the whole project is a reflection of my own personal interests.

In so doing, the first chapters covers the mechanism that linux has that handle execution and process creation and management, what is a process what it sees and what it's universe comprises of is covered, and along with the syscalls and c sections the foundations of the project are set.

Later on we discuss the communication framework that's to be used in the Protocol Buffer and RPC chapter, along with the overall effect on the performance of the system, how they work and how they can be applied.

In a separate chapter XDP is covered, a special project that's part of the eBPF family of programs that's seen recent popularity and applicability on a huge number of projects both commercial and research focused.

Docker and Kubernetes have their own dedicated chapters, along with mentions on collaborating projects CRIU and Karpenter that are used in the project, facilitating process migration and "compute node" discovery.

Finally, we have a theoretical chapter on distributed systems and their theory, major algorithms are being mentioned and their usages on our project explored, accompanied in the end with the implementation chapter: whose sole purpose is to give a high level explanation of the architectural decisions made and the routes taken toward the end goal.

Chapter 2

Containers, Orchestration, and Scaling

The engine of Chapter ?? is correct but static: a fixed set of worker processes started by hand. To make it scale effortlessly — to add capacity by declaring a number rather than by wiring up machines — its workers must become a fleet that some external system starts, replaces, places, and grows automatically. That system is a container orchestrator [1, 3], and in practice it is Kubernetes running containers built with Docker/OCI tooling. This chapter is a thorough, advanced introduction to both, oriented throughout toward one payoff: run behind an orchestrator, the coordinator/worker engine turns into a *horizontally elastic function service* — tasks submitted, scheduled onto an autoscaled worker pool, and executed on demand. Structurally that is a self-hosted *distributed lambda*, and when tasks form a map/shuffle/reduce DAG it is a *MapReduce*. The recurring theme is a boundary: what the orchestrator may own versus what the engine must never delegate [2].¹

2.1 Docker and the container model

In one line: a container is a resource-capped, namespaced process — these are enough to simulate an effectively isolated environment.

Historically we could say that containers are the continuation or adaptation of virtual machines, but this is only true in the way that virtual machines are the replacements of actual server machinery. Containers are however the last executable and writable layer on top of an image, in which all the "work" is distilled, and the biggest difference with virtual machines is that containers rely on the host os to run, that is the host kernel, making it easy to have multiple containers in one computer run, and with the isolation mechanisms having been developed them taking turns executing is under complete isolation from each other. Images need to be created by dedicated teams collecting all application dependencies, and it needs to be in need only of one kernel in order to run, because all programs that are to run on the container will be completely separated from the outside world. Otherwise it is an ordinary Linux process (or process tree) started with a restricted view on the host system.

The most important noun is for sure the *image*, which is the read-only basis of the container, just like the java source is the blueprint of a class in which case the container would be an instance of it - an object -, in actuality it would be a running instance of an image with the container being a writable layer on top of it, with what needs to be persisted needing to be attached to a *volume*. The volume (directory) needs to be mount on the container's filesystem.

Relating, the *bind mount*, maps a specific host directory into the container instead, which also facilitates perfectly development needs. Communication to the outside world happens via the docker network drivers.

¹Companion: Burns, Beda & Hightower, *Kubernetes Up and Running* — used as a conceptual reference, not reproduced.

Containers on Docker’s default ‘bridge’ network can reach the outside world, but container-to-container communication does not provide the convenient name-based discovery available on user-defined networks. Consequently, the first practical setup step is almost always to create a user-defined network, within which containers can refer to one another by name, with Docker providing the necessary DNS resolution.

Docker networks are implemented through different network drivers, each corresponding to a different networking model and use case. The most important drivers are ‘bridge’, ‘host’, ‘none’, ‘overlay’, and ‘macvlan’.

The ‘bridge’ driver is the normal choice when several containers run on the same Docker host. Docker creates a virtual network bridge on the host, and containers attached to it receive their own network interfaces and IP addresses. A user-defined bridge is preferable to Docker’s automatically created default bridge because it provides automatic DNS-based service discovery and gives the administrator explicit control over which containers belong to the network.

The ‘host’ driver removes much of the isolation of Docker’s network. Instead of giving the container its own isolated network stack, the container uses the host’s network namespace directly.

The driver ‘none’ provides essentially no network connectivity to the container, which makes it useful for workloads that do not require networking and where network exposure should be minimized.

The driver ‘overlay’ becomes important when containers are distributed across multiple Docker hosts, allowing them to communicate over a shared logical network spanning those hosts.

Finally, ‘MACvlan’ takes a different approach: instead of making the container appear as an endpoint behind Docker’s virtual networking, it can give the container its own MAC address and make it appear as a device directly connected to the physical network. This is useful for applications that need to participate directly in an existing Layer-2 network.

Configuration enters through environment variables (and, for anything sensitive, files mounted from tmpfs rather than baked into the image, since image layers are readable by anyone who can pull them). In practice these four are created/defined in a Compose file that declares the images, the volumes, the networks and the published ports for a whole set of services in one place, and brings them up and down together, even defining dependencies between containers.

The most important attribute of the container, and the mechanism by which it can be used and considered an analog to a virtual machine is the mechanisms by which we can enforce its isolation: Namespaces restrict what a process can see (PID, mount, network, IPC, UTS, user, and cgroup namespaces, created by clone with the `CLONE_NEW*` flags), cgroupsv2 limit what it can consume (CPU, memory, I/O, and PIDs), seccomp filters restrict which system calls it may make, and dropped capabilities strip privileges otherwise associated with root, which is a main point of podman, which unlike docker, is daemonless and supports rootless containers, allowing a container to run as root internally while that root identity is mapped through a user namespace to an unprivileged host user, thereby avoiding the need for the container runtime to possess host-root privileges, in linux daemons are all root owned. These all show that safety is defined in an application context and for that untrusted code needs to run in a VM-based sandbox (gVisor, Firecracker, Kata).

Three tiers. Docker is not one program but three separable concerns, and almost every property that follows — including every security property — depends on which of the three a mechanism belongs to. The *runtime* is the software that actually runs a container: it talks to the kernel, sets the isolation up, and gets out of the way. The *daemon* sits above it, serving build, start and stop requests and owning the lifecycle of the objects a user names — images, containers, volumes, networks. The *orchestrator* sits above that, deciding where containers run, scheduling them, and reacting to failures. The tiers are defined by interfaces rather than by one vendor’s product, which is why each is independently replaceable.

What the runtime builds. A container is not a virtual machine. It is an ordinary Linux

process (or process tree) started with a restricted view and a bounded appetite, built entirely from the kernel primitives of Chapters ??–??. Two mechanisms *construct* it: *namespaces* restrict what it can see — PID, mount, network, IPC, UTS, user, cgroup and time, created by `clone` with the `CLONE_NEW*` flags — and *cgroups v2* bound what it can consume (CPU, memory, I/O, PIDs). Three more *constrain* it once built: dropped *capabilities* strip root’s powers one by one, a *seccomp* filter allows only the syscalls it actually needs, and a mandatory access control profile (AppArmor’s path-based rules, or SELinux’s contexts) confines it at the kernel’s own boundary. The distinction is worth holding on to: the first two make the container, the last three make it safe. There is no guest kernel; a container shares the host’s, which is why it starts in milliseconds and also why, by default, it is an isolation boundary but not a security boundary against a kernel exploit — untrusted code wants a VM-based sandbox (gVisor, Firecracker, Kata).

The runtime’s division of labour. The Docker CLI talks to a daemon that delegates to `containerd`, which starts a *shim* per container, which invokes `runc`. `runc` sets up the namespaces, cgroups and pivot-root, installs the seccomp filter and capability set, and `execves` the entrypoint per the OCI runtime spec — and then exits, leaving nothing of itself behind. The shim is what remains: it becomes the container’s parent, holds its `STDIN/STDOUT` open and reaps it, which is why a container survives a restart of the daemon above it. Because the interface is the runtime spec, the runtime is replaceable (`crun`, `youki`, or a VM-backed implementation), and Kubernetes talks to the same lower layers through the CRI (Container Runtime Interface), so Docker-the-daemon is not required on a cluster — `containerd` suffices.

What the daemon manages: images. Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlayfs) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once. The practical discipline: order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary), use *multi-stage* builds to keep compilers out of the runtime image, and ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface. Reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

textbfWhat the daemon manages: images. Images package the filesystem and metadata, and stand to containers as a class stands to an object. An OCI image is an ordered set of tarball *layers* unioned at runtime (overlayfs) into a root filesystem, plus a JSON config (entrypoint, env, exposed ports). The daemon does the stacking; what the container sees is a single tree, with one thin writable layer on top that dies with it. Layers are content-addressed, so identical layers are cached and shared across images and pulled once. The practical discipline: order the Dockerfile so rarely-changing layers (dependencies) come before volatile ones (your binary), use *multi-stage* builds to keep compilers out of the runtime image, and ship a *distroless* or `scratch` base — for the statically linked C worker of this project, a few megabytes with almost no attack surface. Reference images by *digest* (`image@sha256:...`) in production, never by mutable tag, so a redeploy cannot silently change the binary.

What the daemon manages: networks and volumes. The same tier owns the two objects that outlive a container. Each container receives its own network namespace and a virtual interface whose far end sits on a host bridge, with routing between them and NAT outward; a published port is a rule mapping a host port inward, and it is the one place the isolation is deliberately opened. That default model puts every container on one L2 segment, which is the structural reason inter-container traffic needs separating rather than trusting. Persistent state arrives by mount: a named volume managed by the daemon, a bind mount of a host path, or `tmpfs` for secrets that should never reach a layer. None of it is distributed — two cells on two hosts sharing a volume means a network filesystem, with all of its consistency behaviour intact.

PID 1 and shutdown. A container's entrypoint runs as PID 1 in its PID namespace, which changes signal semantics (Chapter ??): the kernel delivers no default action for signals to PID 1, and PID 1 must reap orphaned children. So the entrypoint must be the real process (via `exec`), or a tiny init, or `SIGTERM` is ignored and the orchestrator's graceful-stop request is dropped. Correct `SIGTERM` handling — stop accepting work, finish or hand back in-flight tasks, exit — is what turns a worker eviction from a lease-expiry storm into a clean handover, and it is the single highest-value piece of container-awareness the engine needs. Resource limits are cgroup limits: a memory limit yields a deterministic cgroup-OOM kill (a clean, attributable crash), while a CPU limit *throttles* — descheduling the process for the rest of the period — which stalls heartbeats and is the most under-diagnosed cause of false failure detection (watch `cpu.stat`'s `nr_throttled`).

The third tier. Everything above is one host. The orchestrator is the tier that decides which host, keeps the declared number of instances running, and replaces the ones that fail — and because images, networks and volumes are already named, portable objects, it can do so by recreating rather than by moving. That is the subject of the next section, and it is also the boundary of what the container model offers this thesis: relocation at the boundary, not migration of a running computation.

2.2 Kubernetes: the orchestration model

Kubernetes a software to manage and deploy applications, literally an orchestrator, the end product is a cluster that can manage vms, containers. Primarily comprised of two entities (and their constituents): the control-plane and the nodes (data-plane), the management happens declaratively, the desired state is described, and the control is tasked with evaluating the current state, when something is diverging actions are taken to reconcile that. Usually, user-space code will be packaged as a container, and a container will be wrapped in a pod (the smallest deployable unit), next higher-level controllers are going to assist their function across the cores or machines allocated and specified for operation.

The most important moving parts are the control plane nodes:

2.2.1 kube-apiserver (The Central Hub)

What it is: The front door and brain of the control plane. It exposes the Kubernetes REST API.

- **The Only Direct Interface to etcd:** `kube-apiserver` is the only component in the entire cluster that communicates directly with etcd, while every other control plane component (scheduler, controllers) and worker component (kubelet) has to go through the API server. That means that this is not the writer and that after etcd writes the info and metadata about the state it's only readable in one place, this comes with the extremely important advantage of providing one place to enforce concurrency control for readers and writers on the distributed system that's to be managed.
- **Stateless & Scalable:** Because `kube-apiserver` stores no local state (it delegates all state persistence to etcd), you can easily scale it horizontally behind a load balancer for High Availability (HA) - installing more instances of it -.
- **Request Pipeline:** Every request, whether issued via `kubectl` or code, moves through a strict processing pipeline before it is ever persisted, and it takes the form of a verb such as `get`, `list`, `watch`, `create`, `update`, `patch`, or `delete` applied to a resource (or a subresource like `Pods/exec` or `scale`).

1. **Authentication:** Confirms the identity of the caller, using mechanisms like client certificates, bearer tokens, or an external identity provider.
2. **Authorization:** Checks whether that identity is permitted to perform the requested action, typically enforced through Role-Based Access Control (RBAC).
3. **Admission Control:** Runs a chain of controllers that can mutate or validate the object on its way in, for example injecting default resource limits or blocking requests that violate security policy.
4. **Storage:** Finally, the API server writes the resulting state to etcd, persisting the state.

2.2.2 etcd

What it is: A strongly consistent, distributed key-value store based on the Raft consensus algorithm, where functions as the storage of all of the api objects.

- **Cluster Storage of state:** Every single object created in Kubernetes (Pods, Services, CRDs, ConfigMaps) lives inside etcd.
- **Watch Mechanism:** Instead of components constantly polling the database (which degrades performance), etcd supports streaming updates. The API server watches etcd and streams real-time events to the controllers/schedulers.
- **Consistency Over Availability:** etcd prioritizes data consistency over partition tolerance, guarding from 'split-brain' conflicts.

2.2.3 kube-scheduler

What it is: The control plane process responsible for deciding which worker node each newly created Pod should run on, kind of like a scheduler for an os, but without the preemption.

- **Decoupled Decision Maker:** The scheduler never actually starts a container or pulls an image itself; that responsibility belongs entirely to the kubelet running on the chosen node, which is responsible for 'touching' the pods. If it is not able to allocate a position to a node that has just entered the cluster, if we want dynamicity we should use an 'extension' of Kubernetes like the Karpenter or the 'cluster-autoscaler'.
- **Two-Phase Placement Engine:** Placement happens in two distinct passes over the available nodes.
 - **Filtering (Predicates):** The scheduler first eliminates any node that simply cannot host the Pod, for reasons such as insufficient CPU or memory, an incompatible taint, or a missing required node label.
 - **Scoring (Priorities):** Among the nodes that remain, it then ranks each one by fit — for example how evenly workloads are spread across zones, proximity to needed data, or overall resource efficiency — and assigns the Pod to whichever node scores highest. Mainly used for reallocating pods after failures or on scaling.

2.2.4 Cluster Autoscaler / Karpenter

What it is: A component that runs *alongside* the control plane, rather than as one of its core pieces, whose job is to add or remove worker nodes based on demand, not part of the default installation of Kubernetes.

- **The Trigger:** Both tools watch the API server for Pods stuck in the **Pending** state because no current node can satisfy their resource requests, and treat that backlog as a signal that more capacity is needed.
- **Cluster Autoscaler:** The older, more established of the two. It works in terms of pre-defined node groups (an AWS Auto Scaling Group, a GCP Managed Instance Group, etc.) and simply scales those groups up or down, meaning new nodes come in whatever instance size and shape the group was already configured with.
- **Karpenter:** A newer, more flexible alternative that provisions individual nodes directly against the cloud provider's API, without needing pre-defined node groups. It picks the specific instance type, size, and even spot-vs-on-demand mix that best fits the exact Pods waiting to be scheduled, which typically means faster and more tightly bin-packed provisioning.
- **Scaling Down:** Both tools also work in reverse, continuously looking for nodes that are underutilized or empty and safely draining and terminating them to avoid paying for idle capacity.
- **Division of Labor:** Once a new node registers with the API server and is marked **Ready** (via the Node Controller in **cloud-controller-manager**), it becomes just another candidate node, and it is **kube-scheduler** — not the autoscaler — that decides which of the pending Pods actually land on it.

The most important moving parts are the data plane nodes:

2.2.5 Worker Node

What it is: Every worker node runs the same three-piece stack: an agent, a runtime, and a network proxy.

Worker Node

```
|-- kubelet
|-- Container Runtime
|-- kube-proxy
```

2.2.6 kubelet

What it is: The main Kubernetes agent running on every worker node.

- Communicates with the **kube-apiserver** to learn what should be running on its node.
- Receives Pod specifications describing the containers, images, and resources a Pod needs.
- Ensures the assigned Pods are running by starting their containers via the container runtime.
- Monitors containers to confirm they stay in the expected state.
- Reports node and Pod status back to the control plane so it always has an accurate picture.
- Performs health checks and restarts containers when necessary, enabling self-healing.

2.2.7 Container Runtime

What it is: The container runtime is responsible for actually running the containers.

Examples include:

- **containerd** – the most widely used runtime, and the default in most managed Kubernetes offerings.

- CRI-O – a lightweight runtime built specifically for Kubernetes.

The kubelet communicates with the runtime through the Container Runtime Interface (CRI), so any compliant runtime can be swapped in without changing kubelet itself.

2.2.8 kube-proxy

What it is: kube-proxy provides the networking functionality required for Kubernetes Services.

- Maintains network rules on the node, typically using `iptables` or `IPVS`.
- Enables communication with Services by resolving their stable virtual IP to real Pod addresses.
- Routes traffic to the appropriate Pods, load-balancing across all healthy backends.
- Implements Service types such as `ClusterIP` and `NodePort`, giving each a working network behavior.

What it is: Every kind of Kubernetes object shares the same `apiVersion/kind/metadata/spec/status` shape and the same declarative contract — you write `spec`, Kubernetes writes `status`, and a controller continuously reconciles the two. What differs from one kind to the next is what that controller actually *does*: a Pod just runs containers, a Deployment manages rolling updates, a Job runs something to completion. The rest of this section is a reference card for each of the kinds you'll use most, showing how they relate to one another, what operations are available on each, and a minimal YAML example.

2.2.9 How the Kinds Relate

- **Deployment** owns a **ReplicaSet**, which owns **Pods**.
- **CronJob** owns a **Job**, which owns **Pods**.
- **DaemonSet** owns **Pods** (one per node).
- **StatefulSet** owns **Pods** (stable identity and storage).
- **Service** selects **Pods** — it does not own them.
- **ConfigMap** and **Secret** are mounted into **Pods** — again, no ownership.

Solid arrow = owns / creates (garbage-collected together via `ownerReferences`). **Dashed arrow** = selects or mounts into (no ownership, no cascading delete).

2.2.10 Pod

What it is: The smallest deployable unit in Kubernetes — one or more containers that share a network namespace and storage volumes, and are always scheduled together on the same node.

You can:

- **Create** — `kubectl apply -f pod.yaml` → `POST /api/v1/namespaces/{ns}/pods`.
- **Inspect** — `kubectl get/describe pod` → `GET` the Pod resource or its `status` subresource.
- **Exec into it** — `kubectl exec` → `POST .../pods/{name}/exec`.
- **Read logs** — `kubectl logs` → `GET .../pods/{name}/log`.
- **Port-forward** — `kubectl port-forward` → `POST .../pods/{name}/portforward`.
- **Kill it** — `kubectl delete pod` → `DELETE .../pods/{name}`. If it's owned by a `ReplicaSet`, `DaemonSet`, or `StatefulSet`, a replacement appears automatically.

- **Cannot** scale, roll back, or migrate itself — Pods are disposable by design; those behaviors belong to the controllers above it.

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.27
      ports:
        - containerPort: 80
```

2.2.11 ReplicaSet

What it is: A controller that guarantees a fixed number of identical Pods are running at all times, matching Pods purely by label selector rather than by name.

You can:

- **Create** — `kubectl apply -f rs.yaml → POST .../replicasets`.
- **Scale** — `kubectl scale rs -replicas=N → PATCH .../replicasets/{name}/scale`.
- **Delete** — `kubectl delete rs →` cascades to its owned Pods via `ownerReferences`.
- **In practice** you rarely create these directly — a Deployment creates and manages them for you.

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

2.2.12 Deployment

What it is: The standard controller for stateless applications, managing ReplicaSets on your behalf so you get rolling updates, rollbacks, and self-healing without touching Pods directly.

You can:

- **Run / create** — `kubectl create deployment or apply -f deploy.yaml → POST .../deployments`.
- **Scale** — `kubectl scale deployment -replicas=N → PATCH .../deployments/{name}/scale`.
- **Update (rolling deploy)** — `kubectl set image deployment/x c=img:v2 → PATCH spec.template`; a new ReplicaSet is created and scaled up while the old one scales down.

- **Roll back** — `kubectl rollout undo deployment/x` → reverts to a previous ReplicaSet revision.
- **Pause / resume a rollout** — `kubectl rollout pause | resume`.
- **Restart** — `kubectl rollout restart` → patches a pod-template annotation, forcing fresh Pods even with no spec change.
- **Stop / kill** — `kubectl delete deployment` → DELETE ...; cascades to its ReplicaSets and their Pods.
- **“Migrate”** — Kubernetes never live-migrates a running Pod. Moving work off a node means cordonning it (`kubectl cordon`) and draining it (`kubectl drain`), which evicts its Pods so the Deployment’s controller reschedules replacements elsewhere.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
```

2.2.13 StatefulSet

What it is: Like a Deployment, but for workloads that need a stable, unique network identity and stable per-Pod storage that survives rescheduling — think databases and other clustered stateful apps.

You can:

- **Create / scale / delete** — same commands as a Deployment, but Pods are created, scaled, and terminated in order (`web-0`, `web-1`, `web-2`, ..., reverse order on scale-down).
- **Update** — `spec.updateStrategy` is either `RollingUpdate` (default, ordered) or `OnDelete` (you delete Pods manually to trigger their replacement).
- **Stable identity** — each Pod keeps the same hostname and, via `volumeClaimTemplates`, the same `PersistentVolumeClaim` even after being rescheduled to a different node.
- **Delete** — `kubectl delete statefulset` removes the Pods but, by default, leaves the PVCs (and their data) behind.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "web"
  replicas: 3
  selector:
    matchLabels:
```

```

    app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
  volumeClaimTemplates:
    - metadata:
        name: www
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi

```

2.2.14 DaemonSet

What it is: A controller that ensures exactly one copy of a Pod runs on every node (or every node matching a selector) — the pattern used for log collectors, node monitoring agents, and network plugins.

You can:

- **Create** — `kubectl apply -f ds.yaml → POST .../daemonsets`.
- **“Scale”** — not by replica count; the Pod count tracks the number of matching nodes automatically, so it grows or shrinks as nodes join or leave.
- **Update** — `RollingUpdate` (default) or `OnDelete`, same idea as `StatefulSet`.
- **Delete** — `kubectl delete daemonset` removes the Pod from every node.

```

apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: node-exporter
spec:
  selector:
    matchLabels:
      app: node-exporter
  template:
    metadata:
      labels:
        app: node-exporter
    spec:
      containers:
        - name: node-exporter
          image: prom/node-exporter

```

2.2.15 Job

What it is: A controller for run-to-completion workloads — it tracks successful completions rather than keeping Pods alive forever, and retries failed Pods up to a limit.

You can:

- **Run** — `kubectl create job` or `apply -f job.yaml → POST .../jobs`.
- **Control throughput** — `spec.completions` and `spec.parallelism` decide how many Pods must succeed and how many can run at once.

- **Suspend / resume** — `kubectl patch job x -p '{"spec":{"suspend":true}}'` pauses it without deleting it.
- **Stop / kill** — `kubectl delete job -cascade=foreground|orphan` controls whether its Pods are deleted along with it.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migrate
spec:
  completions: 1
  parallelism: 1
  template:
    spec:
      containers:
        - name: migrate
          image: myapp/migrate
      restartPolicy: Never
```

2.2.16 CronJob

What it is: A scheduler that creates a new Job on a cron-style schedule — the Kubernetes equivalent of a crontab entry, but producing real, trackable Job/Pod objects each run.

You can:

- **Create** — `kubectl create cronjob` or `apply -f cronjob.yaml → POST .../cronjobs`.
- **Suspend** — `kubectl patch cronjob x -p '{"spec":{"suspend":true}}'` stops new runs without deleting the schedule.
- **Control overlap** — `spec.concurrencyPolicy: Allow, Forbid, or Replace`.
- **Trigger manually** — `kubectl create job -from=cronjob/x manual-run-1` runs it immediately, outside the schedule.
- **Delete** — `kubectl delete cronjob` removes the schedule and its recent Jobs.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: backup
              image: myapp/backup
          restartPolicy: OnFailure
```

2.2.17 Service

What it is: A stable virtual address that load-balances traffic to whichever Pods currently match its label selector, so clients never need to track individual Pod IPs.

You can:

- **Create** — `kubectl expose deployment/x -port=80` or `apply -f svc.yaml → POST .../services`.

- **Change routing** — editing `spec.selector` or `spec.ports` re-routes live traffic as soon as kube-proxy picks up the updated Endpoints, no Pod restart needed.
- **Choose exposure** — `spec.type`: `ClusterIP` (internal only), `NodePort` (also exposed on every node), or `LoadBalancer` (provisions a cloud load balancer via `cloud-controller-manager`).
- **Delete** — `kubectl delete service` removes the virtual IP/load balancer; it never touches the backing Pods themselves.

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

2.2.18 ConfigMap & Secret

What it is: Two nearly identical objects for injecting configuration into Pods without baking it into the container image — a `ConfigMap` for plain values, a `Secret` for sensitive ones (base64-encoded at rest, and typically further protected by encryption-at-rest and RBAC).

You can:

- **Create** — `kubectl create configmap/secret` or `apply -f → POST .../configmaps` or `.../secrets`.
- **Consume** — mount as a volume (files appear under a directory) or inject as environment variables via `envFrom/valueFrom`.
- **Update** — `kubectl edit` or `apply → PATCH ...`; volume-mounted files update on the kubelet's next sync automatically, but env-var injections require the Pod to restart to pick up the change.
- **Delete** — `kubectl delete configmap/secret`; Pods already using the old data keep running until they're restarted.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
data:
  DB_PASSWORD: cGFzc3dvcmQ= # base64-encoded
```

2.3 The object model

, and where each fits the engine: This what the context of kubernetes encompasses, defined below are the nouns and verbs of kubernetes, making it clear what actions and objects we can handle for the management of the lifecycle of our userspace programs.

Pod	one+ co-scheduled containers sharing net+IPC ns (localhost, sidecars)	
Deployment	N interchangeable STATELESS pods, rolling updates	-> WORKERS
StatefulSet	stable names/storage, ordered ops	-> COORDINATOR quorum
Job / CronJob	run-to-completion / scheduled batch	-> occasional batch only
DaemonSet	one pod per node	-> node agents
Service	stable virtual IP + L4 LB over a pod set	
Headless Svc	one DNS A-record per pod (no VIP)	-> peer discovery
ConfigMap/Secret	configuration / credentials	
PVC / PV / StorageClass	durable volumes via CSI	-> logs, DB state
Ingress / Gateway	north-south HTTP entry	-> Submit front door
HPA / VPA	autoscale replicas / right-size requests	-> scale workers
PodDisruptionBudget	cap voluntary disruption	-> protect quorum
NetworkPolicy / RBAC / SA / securityContext	least privilege	
CRD + controller	extend the API with your own object + reconcile loop	

Buy consensus; do not autoscale it. A quorum's size is a correctness parameter, not a capacity knob — an HPA must never resize the coordinator StatefulSet, because adding a member is a membership-change protocol (Chapter ??), not a replica bump. And run *your own* etcd: sharing the cluster's control-plane etcd couples your load to the API server's health, a documented way to break a cluster.

Durability on Kubernetes has its own caveats: a PVC is not a backup (snapshot to object storage and *test restore*); the log's device must give low-latency **fsync**, so prefer local NVMe over network storage whose sync is a network round trip.

2.4 API gateways, ingress, and service mesh

Traffic splits into *north-south* (clients → cluster) and *east-west* (service → service), and each has an edge.

North-south is the Ingress / Gateway API and its controller (Envoy, NGINX, Traefik): one entry point that terminates TLS, routes by host/path, enforces authentication and rate limits, and splits traffic for canary or blue-green rollouts. For the engine this is the *Submit* front door: it authenticates callers, applies per-tenant rate limits and load shedding (Chapter ??), and routes to the coordinator leader — so the admission and backpressure logic lives at a single, observable choke point rather than being scattered.

East-west is optionally a *service mesh*: a sidecar proxy (Envoy) injected beside every pod that transparently adds mutual TLS (service identity), retries, timeouts, circuit breaking, and per-call telemetry. This is exactly the fault-tolerance and RPC-hardening machinery of Chapter ??, *externalized* out of the application into the platform. The trade is real overhead (a proxy hop and CPU per pod) and operational complexity, so a mesh earns its place at scale or across trust boundaries, not by default; a C engine that already implements deadlines, retries with budgets, and mTLS in its own RPC layer may reasonably skip it and keep the control it already has.

Chapter 3

Two Transports: XDP and gRPC

The control-plane/data-plane split that runs through this thesis (Chapter ??) has a transport at each end, and they pull in opposite directions. The *data plane* — bulk task payloads, results, checkpoint images moving between nodes — wants the lowest possible per-byte and per-packet cost, even at the price of building reliability by hand. The *control plane* — **Submit**, dispatch, status, the worker channel — wants typed, evolvable, reliable request/response with deadlines and streaming, and to fit the Kubernetes ecosystem of Chapter 2. This short chapter presents one technology aimed squarely at each: **XDP** for the hot data path in the kernel, and **gRPC** for the application-layer control path.

3.1 XDP: programmable packet processing at the driver

XDP (eXpress Data Path) [5, 4] runs an *eBPF* program at the earliest point in the receive path — inside the NIC driver, before the kernel allocates an `sk_buff`, and on capable hardware offloaded onto the NIC itself. eBPF is sandboxed bytecode: the kernel *verifier* proves the program terminates and touches only permitted memory, then JITs it to native code, so it runs at kernel speed without the ability to crash the kernel; persistent state lives in *maps* shared with userspace. An XDP program sees each raw frame and returns a verdict: **XDP_DROP** (discard — the basis of line-rate DDoS filtering), **XDP_PASS** (hand up to the normal stack), **XDP_TX** (bounce back out the same NIC), or **XDP_REDIRECT** (send to another NIC, another CPU, or a userspace socket). Because it acts before the sockets/TCP machinery exists, it is where an L4 load balancer (spreading **Submit** or dispatch across coordinators and workers) or a custom router belongs when the per-packet cost of the full stack is the bottleneck.

The piece most relevant to this project’s data plane is **AF_XDP**: a socket type that, paired with an XDP **REDIRECT**, DMA’s frames directly into a userspace memory region (a *UMEM* of fill/completion rings) with *zero copy* and no traversal of the kernel network stack. This is kernel-bypass packet I/O reachable from an ordinary process — the mechanism behind userspace networking frameworks — and it is the natural substrate for a specialized, connectionless bulk transfer of results or checkpoint images between nodes at line rate. The cost is that you inherit everything the stack was doing for you: **AF_XDP** delivers *frames*, not a reliable ordered byte stream, so segmentation, acknowledgement, retransmission, and congestion control are yours to build — which is precisely the RPC-from-first-principles exercise of Chapter ??, now at the packet level. Combined with the verifier’s limits on program size and complexity and the need for driver support, this makes XDP/**AF_XDP** a tool for a small number of hot paths — a load-balancing front end, or a bulk data mover — not a general RPC layer.

3.1.1 eBPF, maps, and the verifier

An XDP program is an *eBPF* program: restricted C compiled by `clang` to a sandboxed bytecode, loaded through the `bpf()` syscall, checked by the in-kernel *verifier*, and JIT-compiled to native code. The verifier is what makes running code in the datapath safe, and it shapes how these programs are written: it proves the program halts (loops must be bounded) and that every memory access is in range, so a frame is parsed by a strict *check-then-read* walk — a header pointer is compared against the frame’s `data_end` *before* it is dereferenced — under a small stack and a bounded instruction budget. Persistent and shared state lives only in *maps*: typed key/value tables the program shares with userspace (through `bpf()`) and across CPUs. Four map types carry this project’s datapath: a `BPF_MAP_TYPE_HASH` routing map (`function_id` → cell), an `ARRAY` of per-function counters, and the two redirect targets `CPUMAP` (a pinned core) and `XSKMAP` (an `AF_XDP` socket). The program reaches the kernel only through *helper functions* — `bpf_map_lookup_elem`, `bpf_redirect_map`, and so on — some of which, redirection among them, require the program to declare a GPL-compatible license.

3.1.2 Attachment modes and the reality of zero copy

The same program can attach in three ways, trading portability for cost. In *offloaded* mode it runs on a capable SmartNIC; in *native* (driver) mode it runs inside the driver’s receive poll, before any `sk_buff` exists — the intended mode, and the only one in which the NIC can DMA a frame straight into userspace memory; in *generic* (SKB) mode it runs later, in the stack after the `sk_buff` is built, which works on any interface (including the `veth` pairs used inside containers) but copies. The distinction is decisive for the zero-copy claim: true zero copy requires native mode *and* an `XDP_ZEROCOPY` socket binding on a driver that supports it, whereas the portable fallback — `XDP_COPY` in generic mode — exercises exactly the same routing logic but pays one copy. Where the benefit is real is therefore a property of the driver and mode, not of the program.

3.1.3 AF_XDP: the UMEM and its four rings

`AF_XDP` couples the `XDP_REDIRECT` verdict to a userspace socket through a shared memory region, the *UMEM*: a block of process memory, registered with the kernel and carved into fixed-size frames, that in native mode is the NIC’s DMA target so packets land in it with no copy. Four single-producer/single-consumer descriptor rings coordinate ownership. Two belong to the *UMEM*: **FILL** (userspace hands the kernel empty frames to receive into) and **COMPLETION** (the kernel returns frames whose transmission has finished). Two belong to the socket: **RX** (the kernel hands userspace descriptors of filled frames) and **TX** (userspace hands the kernel frames to send). A frame’s life is thus a cycle `FILL` → (DMA) → `RX` → processing → `FILL`. The bridge from the in-kernel verdict to one specific socket is the `XSKMAP`: userspace registers each socket’s file descriptor at its receive-queue index, and `bpf_redirect_map(&xsks_map, queue_id, ...)` in the program lands the frame in that socket’s `RX` ring. Because the rings are per queue, the design scales by binding *one socket per NIC receive queue* (spread by the hardware’s receive-side scaling) and pinning each to a core, which removes any shared reader and maps one-to-one onto a per-core worker pool; an optional `need_wakeup` flag lets the kernel signal when a `poll()` is actually required, amortizing syscalls at high rates.

3.1.4 How this project uses XDP

In our design XDP is the *routing plane* for the job data path, not a general transport. Each record travels in a UDP frame with a small fixed header (magic, `function_id`, `feed_id`, `record_id`, length) placed before the struct payload. An XDP program on the receiving node parses *only that header* — never the struct — reads `function_id`, looks it up in an eBPF *routing map* (`function_id` → target cell), and `XDP_REDIRECTs` the frame, either to a `CPUMAP` (a pinned core)

or, for zero copy, into the target cell’s **AF_XDP** socket, before the kernel builds an **sk_buff**. The cell’s worker reads frames from its AF_XDP RX ring, hands the payload to its thread pool, runs the **dlopen**’d function, and writes results back through the TX ring. Two properties make this the right tool here: routing is *format-agnostic* (the payload’s struct/FlatBuffer layout stays owned by the function; XDP touches only the header), and *relocation is a single map write* — when a cell moves to another node, updating **function_id** → target makes the stream follow it. The prototype in **experiments/xdp/** implements the parse-and-classify half (per-function counters in an eBPF map, driven by a UDP feeder); the production step replaces the counter with the **XDP_REDIRECT** into AF_XDP. Consistent with Chapter ??’s RPC-from-first-principles caveat, we use XDP/AF_XDP only on the hot, small-record path and keep ordinary sockets or gRPC for control and moderate streams.

3.1.5 Frame ownership across the pool, and in-kernel admission

Two consequences of the UMEM model matter once frames feed a thread pool rather than a single reader. First, ownership outlives the RX ring: a worker reads its job *in place* from the UMEM frame, so that frame cannot return to the FILL ring until the worker finishes; the reader records each frame’s address alongside the job and recycles it only after **run()** completes, and because several workers recycle into the single-producer FILL ring, those returns are serialized. If every frame is in flight the FILL ring drains and the NIC drops — a natural back-pressure bound set by the frame count and the queue depth. Second, the program can reject work *before* it costs anything: validating the header’s magic and destination in-kernel and returning **XDP_DROP** for non-conforming frames discards traffic aimed at the data-plane port before an **sk_buff**, a wakeup, or an allocation — the same mechanism that makes XDP a line-rate DDoS filter, here serving as admission control for the job stream, while conforming traffic on other ports is passed up untouched with **XDP_PASS**.

3.1.6 Deployment: privileges and containers

Because XDP is a kernel facility and containers share the host kernel, running the router or an AF_XDP receiver in a container is a question of privilege and namespace, not of a special runtime. Attaching the program needs **CAP_NET_ADMIN**; loading it and its maps needs **CAP_BPF** (or **CAP_SYS_ADMIN** on kernels before 5.8); the UMEM and maps are locked memory, so **RLIMIT_MEMLOCK** must be raised; and a runtime’s default seccomp policy typically blocks the **bpf()** syscall unless one of those capabilities is present. The program attaches to an interface *in the container’s network namespace*: on a bridge network that is a **veth** endpoint, which supports only generic mode and hence the copy fallback, whereas true zero copy requires a native-XDP NIC in the namespace — host networking, or an SR-IOV virtual function passed in, the arrangement the Kubernetes SR-IOV and AF_XDP device plugins provide. This is the same portability-versus-performance boundary as the attachment modes above, drawn now at the container edge.

3.2 gRPC: typed RPC on HTTP/2

gRPC is the opposite choice: a batteries-included, application-layer RPC framework and the de facto standard for service-to-service calls on Kubernetes. Interfaces and messages are declared in *Protocol Buffers*, whose compiler generates client and server stubs and whose tag-based binary encoding gives compact wire size and the backward/forward schema evolution of Chapter ?? (add optional fields, never reuse a tag). The transport is HTTP/2, which supplies — off the shelf — exactly the mechanisms that chapter built by hand: *multiplexed streams* over one connection, per-stream *flow control*, and header compression. Four call shapes fall out of it: unary, server-streaming, client-streaming, and *bidirectional streaming* — the last being the ideal form for the

long-lived worker channel on which the coordinator dispatches tasks and the worker streams heartbeats and completions.

Around this core gRPC provides the production concerns as first-class features: *deadlines* propagated in call metadata (so a client’s deadline bounds all downstream work), cancellation, status codes carrying retryability, pluggable *interceptors* for authentication, tracing, and metrics, and name resolution with client-side load balancing. In a cluster it slots directly into the Chapter 2 machinery: it traverses Ingress and Gateway controllers, gets mutual-TLS identity and retries from a service mesh for free, and has a standard health-checking protocol probes can use. For this project it is the natural control-plane transport — client→gateway→coordinator, and the coordinator↔worker channel — giving multiplexing, deadlines, streaming, and schema evolution without hand-rolling them. Its costs place it on the control side of the split: HTTP/2 framing and HPACK cost CPU at very high message rates, and because streams share one TCP connection a lost segment head-of-line-blocks all of them (a limitation HTTP/3 over QUIC removes), so gRPC is the wrong tool for the XDP-class bulk data plane.

3.3 Where each fits

	XDP / AF_XDP	gRPC
plane	data (bulk, hot path)	control (API, dispatch)
layer	L2-L4, pre-stack / kernel-bypass	L7 on HTTP/2
unit	raw frames	typed messages + streams
reliability	you build it	provided (over TCP)
evolution	n/a (custom)	protobuf schema, tag-based
k8s fit	CNI dataplanes, L4 LB, offload	Ingress, mesh, mTLS, health
use here	result/checkpoint bulk transfer, in-cluster load balancing	Submit/status, worker channel, inter-component RPC

The two are complementary, not competing: they are the concrete realizations of the same control/data-plane split. gRPC carries the small, typed, reliable control traffic that must be correct and evolvable; XDP/AF_XDP (or, more modestly, plain TCP plus the object store) carries the large, latency-sensitive data traffic that must be fast. Choosing per plane, rather than forcing one transport everywhere, is the same discipline — match the mechanism to the guarantee it must provide — that governs every layer of this system.

Chapter 4

Protocol Buffers

The gRPC control plane of Chapter 3 rests on **Protocol Buffers** (protobuf) [6]: Google’s interface-definition language and binary wire format for structured data. It is the concrete realization of the schema-compatibility discipline this thesis returns to repeatedly — both ends must agree on a layout, and disagreement must be caught rather than tolerated (Chapter ??’s `point.h` static assertions were the toy version). This chapter explains protobuf as a technology and then, in Section ??, exactly where it does and does not appear in this project — a distinction that matters, because the data plane deliberately uses a different, zero-copy cousin of protobuf rather than protobuf itself.

4.1 The schema and code generation

A protobuf schema is a `.proto` file: a set of *messages*, each a list of named, typed fields, and — decisively — each field carries an integer *field number* (its tag). The schema is the single source of truth; the compiler `protoc` generates, for each target language, the type together with its accessors and its (de)serialization code, so every participant speaks the same format without hand-written parsing. In `proto3` there are no required fields (all are optional with well-defined defaults), and the type system covers scalars (integers of several widths and signednesses, `bool`, `double`, `float`, `string`, `bytes`), `enums`, nested messages, `repeated` fields (lists), `maps`, and `oneof` (a tagged union). The field *name* is a source-level convenience; on the wire only the *number* identifies a field, which is the root of protobuf’s compactness and its evolution story.

4.2 The wire format

An encoded message is a flat concatenation of fields, each a key followed by a value. The key is a single varint packing the field number and a 3-bit *wire type*, `key = (field_number << 3) | wire_type`, and the wire type tells a reader how to consume the value even for a field it has never heard of:

wire type 0	varint	int32/64, uint32/64, bool, enum (sint via ZigZag)
wire type 1	64-bit	fixed64, sfixed64, double
wire type 2	length-delim	string, bytes, embedded messages, packed repeated
wire type 5	32-bit	fixed32, sfixed32, float

Varints store an integer seven bits per byte, the high bit marking continuation, so small numbers cost one byte; signed values use ZigZag (`sint32/64`) so that $-1, 1, -2, 2 \dots$ map to $1, 2, 3, 4 \dots$ instead of producing ten-byte negatives. A length-delimited field is a varint length then that many bytes, which is how strings, opaque `bytes`, and nested messages nest uniformly. Concretely, a message `{ function_id = 7; payload = AA BB CC }` encodes as:

```

field 1 (function_id=7): 08 07
    08 = (1<<3)|0 (field 1, varint)    07 = value 7
field 4 (payload, 3 bytes): 22 03 AA BB CC
    22 = (4<<3)|2 (field 4, length-delim) 03 = length  AA BB CC = bytes

```

Two properties follow directly. Fields are *order-independent and self-delimiting*: a reader keys off the field number and skips unknown fields using the wire type, so the message is not positional. And field *names never travel* — the wire carries numbers and values only — which keeps it compact but means the `.proto` is mandatory to interpret the bytes. This is the exact opposite of the data plane’s flat buffer, where fields live at *fixed offsets* with no tags and are read in place with no decode (Section ??).

4.3 Schema evolution

Because a field is identified by number and unknown fields are skippable, a schema can change without breaking peers built against an older or newer version, provided a few rules hold: a field number, once used, is never reused or repurposed; new fields take new numbers and, being optional, default cleanly when absent; and retired numbers/names are **reserved** to prevent accidental reuse. Old readers ignore (or, in many implementations, preserve) fields they do not know; new readers see defaults for fields an old writer omitted. This is precisely the “add optional fields, never reuse a tag” backward/forward compatibility that Chapter 3 credited to the control plane, and it is what lets a running cluster be upgraded component by component rather than in lockstep.

4.4 Implementing the wire format

The previous sections describe the format. This one reports what implementing it teaches. The codec behind this project’s control plane is written by hand — `cluster/src/pb.c`, about two hundred lines, no dependencies — rather than generated by `protoc`. In any other setting that would be the wrong choice: generated code is correct, complete and free.

What generation hides is that the properties attributed to Protocol Buffers in §4.3 follow from two decisions visible in twenty lines of a decoder.

4.4.1 Forward compatibility is one line

Every decoder in `msg.c` has the same shape:

```

while (pb_read_tag(&r, &field, &wire)) {
    if (!wire_legal(wire)) return -1;
    switch (field) {
        case 1: m->function_id = (uint32_t)pb_read_varint(&r); break;
        ...
        default: if (pb_skip(&r, wire) < 0) return -1;          break;
    }
}

```

The `default` arm is schema evolution. Not a library feature, a negotiation or a version handshake: a `switch` default that steps over what it does not recognise. A worker compiled against last month’s `.proto` decodes this month’s coordinator correctly, ignoring what it cannot name. It can step over those fields because the wire type travels in the tag, so the length of a value is computable without knowing what the value means. “Add fields freely, never reuse a tag number” is not a convention enforced by tooling; it is the condition under which that one line is sound.

The flat buffer’s counterpart is the absence of the loop. No tags, so nothing to skip, so no evolution. Two hundred lines of decoder against zero is the trade.

4.4.2 Default elision, and what it costs

A proto3 scalar equal to its default never reaches the wire, so every field writer starts with a test:

```
void pb_put_uint32(pb_writer *w, uint32_t field, uint32_t v)
{
    if (!v) return;                /* absent == zero, on purpose */
    pb_put_tag(w, field, PB_VARINT);
    pb_put_varint(w, v);
}
```

Hence a message whose fields all hold defaults encodes to zero bytes, a case the conformance vectors check. There is a real loss here: for a proto3 scalar, “absent” and “present and zero” are the same bytes and no reader can tell them apart. Proto2 had explicit presence, proto3 removed it, and later versions restored it only for `optional` and message-typed fields.

This shapes schema design directly, and the project’s schema shows it. Status codes put zero on success and every failure on a non-zero value, because a status using zero for a failure mode would be indistinguishable from an absent status. That is forced by the encoding, and it is the sort of coupling between wire format and schema that a generated codec makes easy to miss.

4.4.3 The fiddly parts

Three details account for most of the difficulty, and a naive version of each is silently wrong rather than obviously broken.

Submessage lengths precede bodies of unknown length. A length-delimited field is a varint length then that many bytes, so encoding a nested message needs its size before writing it. The usual answers are to encode twice, or to encode into scratch space and prepend. This implementation reserves a two-byte placeholder, writes the body, computes the true varint width and `memmoves` if they differ. Correct at any size, with one small move in the uncommon case. Two bytes covers bodies up to 16,383, which is every control-plane message here, so the move never happens in practice — and the vectors include a 200-byte submessage to exercise the branch that would otherwise go untested.

Varints have a maximum length, and exceeding it is undefined behaviour rather than merely wrong. A 64-bit value needs at most ten bytes ($\lceil 64/7 \rceil$). A decoder still accumulating on an eleventh shifts by seventy bits, and a shift count at least as wide as the promoted type is undefined in C.¹ So malformed input is a language-level problem, not a data problem, and the check that rejects it is required for correctness rather than defensive.

Every length off the wire is attacker-controlled. The reader bounds each one before use, and writes each comparison as a subtraction, `1 > r->len - r->pos` rather than `r->pos + 1 > r->len`. The addition can wrap and a wrapped comparison passes.² The same rule governs flat-buffer offset validation on the data-plane side, which is the more dangerous of the two: a flat buffer is dereferenced, not copied.

¹ISO/IEC 9899:2011 §6.5.7p3.

²CERT INT30-C.

4.4.4 Checking the claim

An implementation that merely resembled Protocol Buffers would pass any test written against itself. So the claim is checked against an independent encoder: `cluster/test/gen_vectors.py` implements the same specification, in another language, without reference to the C, and `make check` compares the two byte for byte across nine messages. Two implementations derived independently from a specification agreeing tells you something about the specification. An implementation agreeing with its own test tells you it is deterministic.

The vectors include the awkward cases on purpose: an all-default message at zero bytes, a maximal ten-byte varint, negative `int64` values widened to full two's complement — the regime `sint64` and ZigZag exist for, and where choosing `int64` costs ten bytes a field — the two-byte submessage prefix, and four malformed shapes that must be rejected rather than survived. A last test appends fields with numbers the build has never seen and confirms the known ones still decode, reducing §4.3 to an assertion.

4.5 gRPC and Protocol Buffers together

The same `.proto` that defines the messages also defines the *service*: a set of `rpc` methods, each naming a request and response message and optionally marking either as a `stream`. `protoc` with the gRPC plugin generates the client stub and server skeleton, and gRPC carries length-prefixed protobuf messages over HTTP/2, supplying the multiplexing, flow control, deadlines, and status codes that Chapter 3 described. The four call shapes (unary, server-, client-, and bidirectional-streaming) are just which sides are marked `stream`; the wire encoding of each message is the format above.

Bibliography

- [1] B. Burns, J. Beda and K. Hightower, *Kubernetes: Up and Running*, O'Reilly Media.
- [2] B. Burns, *Designing Distributed Systems*, O'Reilly Media, 2018.
- [3] N. Poulton, *The Kubernetes Book*, self-published.
- [4] B. Gregg, *BPF Performance Tools*, Addison-Wesley, 2019.
- [5] L. Rice, *Learning eBPF*, O'Reilly Media, 2023.
- [6] Google, *Protocol Buffers Documentation*. <https://protobuf.dev/>